

Project Report: Document Classification System for Sports vs. Politics

1. Data Collection and Methodology

1.1 Data Source Acquisition

The foundation of this project is the **HuffPost News Category Dataset (v3)**. While many NLP projects use pre-processed libraries (like `nltk.corpus`), this task required manual data handling to ensure a deep understanding of data pipelines.

- **Platform:** Kaggle.
- **Original URL:** <https://www.kaggle.com/datasets/rmisra/news-category-dataset>
- **Collection Method:** The raw data was downloaded in a compressed `.json` format. This format is "JSON-Lines," meaning each line is an independent JSON object.

1.2 Data Filtering and Extraction

The original dataset contains 209,527 news headlines across 42 categories. For the scope of this binary classification task, a custom Python extraction script was written to:

1. Iterate through the JSON-Lines file.
2. Filter only for records where the `category` key matched `SPORTS` or `POLITICS`.
3. Concatenate the `headline` and `short_description` fields to maximize the feature set for each entry.
4. Standardize labels to uppercase for consistency.

2. Dataset Description and Analysis

2.1 Feature Set

Each data point in our refined dataset consists of a text string (the combined headline and description) and a categorical label.

- **Total Filtered Samples:** ~40,000 (Subsampled to 2,000 for local computational efficiency).
- **Class Distribution:** The dataset is naturally imbalanced in the HuffPost corpus (Politics significantly outweighs Sports), but for this project, a stratified sampling approach was used to maintain a near 50/50 split to avoid model bias.

2.2 Text Analysis (Preprocessing)

Before feeding text into mathematical models, it must be normalized. Our analysis of the raw text revealed significant noise (HTML tags, emojis, and varied casing). The following pipeline was implemented:

- **Case Normalization:** All text converted to lowercase to ensure "Election" and "election" are treated as the same feature.
- **Punctuation Stripping:** Characters such as `!`, `?`, and `.` were removed as they rarely contribute to topic-specific sentiment in this context.
- **Tokenization:** The text was split into individual words (tokens) based on whitespace.

3. Feature Representation (Bag of Words)

Since machine learning models cannot process raw strings, a **Bag of Words (BoW)** model was built from scratch.

1. **Vocabulary Fitting:** The algorithm scans the training set to find every unique word, assigning each a specific index in a master dictionary.
2. **Vectorization:** Every sentence is transformed into a fixed-length vector where the value at index i is the count of how many times the i^{th} word of the vocabulary appeared in that sentence.
 - *Example:* If the vocabulary is [ball, vote, win], the sentence "vote for the win" becomes [0, 1, 1].

4. Machine Learning Techniques

This project focused on three distinct paradigms of machine learning: Probabilistic, Discriminative, and Instance-based.

4.1 Naive Bayes

Naive Bayes is based on the application of Bayes' Theorem with the "naive" assumption of conditional independence between every pair of features.

- **Formula:** $P(C|d) \propto P(C) \prod_{i=1}^n P(w_i|C)$
- **Implementation:** We calculated the log-likelihood of each word belonging to a class. **Laplace Smoothing** was added to the denominator to prevent a probability of zero when a word from the test set does not exist in the training set.

4.2 Logistic Regression

This is a linear model that uses the sigmoid function to map any real-valued number into a probability between 0 and 1.

- **Sigmoid Function:** $\sigma(z) = \frac{1}{1 + e^{-z}}$
- **Optimization:** Unlike library-based versions, we implemented **Batch Gradient Descent**. We initialized weights at zero and updated them by calculating the derivative of the Cross-Entropy loss function over 500 iterations.

4.3 K-Nearest Neighbors (KNN)

KNN is an instance-based learner. It does not "learn" a model; instead, it stores all training vectors.

- **Metric: Euclidean Distance.** $d(p, q) = \sqrt{\sum (p_i - q_i)^2}$
- **Mechanism:** For a new headline, the system calculates the distance to all 1,600 training headlines and picks the most common label among the 5 closest neighbors ($k=5$).

5. Quantitative Comparisons

The following results were achieved using a 20% test split:

Metric	Naive Bayes	Logistic Regression	KNN (k=5)
Accuracy	92.00%	89.25%	89.00%
Training Speed	Instant	Moderate (500 epochs)	Instant
Prediction Speed	Very Fast	Very Fast	Very Slow
High-Dim Performance	Excellent	Good	Poor

Comparative Analysis

- **Naive Bayes** performed best because news topics are often dictated by specific "trigger" words (e.g., "stadium" or "senate"). The independent nature of the model allows it to pick up these signals regardless of sentence structure.
- **KNN** struggled due to the **"Curse of Dimensionality."** In a vocabulary of 5,000+ words, most vectors are "sparse" (mostly zeros), making the distance between any two documents very similar and less meaningful.

6. Limitations of the System

Despite the high accuracy of Naive Bayes, the system has several inherent flaws:

1. **Loss of Context:** The Bag of Words model ignores word order. The phrase "The team did not win the election" might be misclassified as Sports because of the word "team" and "win," even though it is about politics.
2. **Computational Bottleneck:** The current KNN implementation is $O(N \cdot D)$, where N is the number of samples and D is the vocabulary size. This makes it impossible to scale to the full 200,000-row dataset without advanced data structures like KD-Trees.

3. **Vocabulary Sensitivity:** The model is highly dependent on the quality of the training vocabulary. If the training data is from 2012, it might fail to recognize 2024 political terms like "TikTok Ban" or "Generative AI policy."

7. Conclusion

This project demonstrates that while complex deep learning models (like Transformers) are the industry standard, classical algorithms like Naive Bayes remain incredibly powerful for topic-based classification. By building these from scratch, we successfully bypassed the "black box" of modern libraries and gained a granular understanding of how text is converted into mathematical probability.
